

GUIA DE ESTUDO

Trabalho de Automação de Testes

API Petstore + Web SauceDemo

Bruno Ibiapina

Maio de 2026

Como usar este guia

Este guia foi escrito pra você ler antes da apresentação e ter respostas prontas pra qualquer pergunta. A apresentação tem no máximo 15 minutos, então o foco é mostrar domínio do conteúdo, não cobrir tudo.

Recomendação de estudo:

- Leia uma vez de ponta a ponta pra ter visão geral.
- Releia os capítulos 4 e 5 (API e Web) com o código aberto ao lado, para casar conceito com prática.
- Decore a estrutura do roteiro de apresentação (capítulo 11) - especialmente os 5 minutos iniciais.
- Treine respondendo as 30 perguntas do capítulo 10 em voz alta.
- No dia, abra o repositório no GitHub e tenha as pipelines rodando ao vivo, isso impressiona.

1. Visão geral do trabalho

1.1 O que foi pedido

A professora pediu duas automações distintas, integradas a uma pipeline de CI:

- **Automação de API** sobre a Swagger Petstore (<https://petstore.swagger.io/v2>), cobrindo os endpoints de Pet, Store e User.
- **Automação Web** sobre o site SauceDemo (<https://www.saucedemo.com>), com fluxo E2E de login, carrinho e checkout, usando Selenium em Python ou Java.

CrITÉRIOS avaliados:

- Qualidade do código (organização e reusabilidade)
- Estratégia de testes (escolha de cenários e validações)
- CI/CD (execução automática via pipeline)
- Domínio do conteúdo (capacidade de explicar a solução)

1.2 O que foi entregue

Um repositório único contendo dois projetos independentes, cada um com sua própria pipeline. A estrutura é:

```
Trabalho-Teste/
├── .github/workflows/      (3 pipelines: api-tests, web-tests, lint)
├── api/                   (testes da API Petstore)
│   ├── clients/           (BaseClient + Pet, Store, User clients)
│   ├── schemas/           (JSON Schemas para validação)
│   ├── data/factories.py   (geração de massa via Faker)
│   ├── tests/             (test_pet, test_store, test_user)
│   ├── config.py
│   └── pytest.ini
├── web/                   (testes Web SauceDemo)
│   ├── pages/             (Page Objects: Login, Inventory, Cart, Checkout)
│   ├── tests/             (test_login, test_cart, test_checkout)
│   ├── utils/driver_factory.py
│   ├── config.py
│   └── pytest.ini
├── Makefile               (atalhos de comandos)
├── README.md
└── TEST_STRATEGY.md
```

1.3 Números do trabalho

Item	Quantidade / Detalhe
Testes de API	23 (9 Pet, 6 Store, 8 User)

Testes Web	13 (5 Login, 5 Carrinho, 3 Checkout)
Pipelines	3 workflows do GitHub Actions
Page Objects	5 (Base + Login + Inventory + Cart + Checkout)
Clientes HTTP	4 (Base + Pet + Store + User)
JSON Schemas	5 (Pet, PetList, Order, Inventory, User)

2. Conceitos fundamentais de QA

2.1 Pirâmide de testes

Modelo proposto por Mike Cohn que descreve a proporção ideal de testes em um software:

- **Base — Testes unitários:** rápidos, isolados, testam uma função/classe. Devem ser a maioria.
- **Meio — Testes de integração / API:** testam a comunicação entre módulos ou serviços externos. Quantidade média.
- **Topo — Testes E2E (UI):** simulam o usuário no navegador. Lentos e frágeis, devem ser poucos.

Por que essa forma? Quanto mais alto na pirâmide, mais lento, mais caro de manter e mais "flaky" (instável). Por isso a base deve ser larga.

No nosso trabalho: como testamos sistemas externos (API pública e site público), não temos camada unitária. Temos a camada de API (média) e a camada E2E (topo).

2.2 Tipos de testes presentes no trabalho

Tipo	Definição
Smoke	Testes mínimos que validam que o sistema "sobe". Rápidos, rodam em todo PR.
Regression	Suite completa que garante que nada quebrou após uma mudança.
Negative	Testam comportamento em cenários de erro (404, campo vazio, valor inválido).
E2E	"End-to-end" — simulam o fluxo completo do usuário. No web, é login → carrinho → checkout.
Contract	Validam o formato (schema) da resposta. Garantem que a API não mudou contrato.

2.3 Caixa preta vs caixa branca

- **Caixa preta (black box):** testa apenas entradas e saídas, sem conhecer o código interno. É o que fazemos aqui — não temos acesso ao código do Petstore nem do SauceDemo.
- **Caixa branca (white box):** testa o código por dentro, com conhecimento da implementação. Típico de testes unitários.

2.4 Conceitos importantes para a apresentação

- **Asserção (assert):** verificação que decide se o teste passa ou falha. Ex.: `assert response.status_code == 200.`

- **Fixture:** preparação de dados/estado antes de um teste e limpeza depois. No pytest, usamos o decorator `@pytest.fixture`.
- **Setup / Teardown:** rotinas que preparam o ambiente antes do teste e limpam depois.
- **Flakiness (teste flaky):** teste que às vezes passa, às vezes falha sem mudança de código. Causa #1 da perda de confiança em automação.
- **Mock / Stub:** substituir uma dependência real por uma falsa para isolar o que se quer testar. Não usamos aqui porque o objetivo é justamente testar o sistema real.
- **Idempotência:** uma operação é idempotente quando executá-la várias vezes produz o mesmo efeito. GET, PUT e DELETE são idempotentes; POST não é.

3. Stack tecnológica - o porquê de cada escolha

3.1 Por que Python?

Justificativas para usar quando perguntada:

- Linguagem extremamente legível, ideal para apresentar.
- Ecossistema de testes muito maduro: pytest é referência mundial.
- Bibliotecas de API (requests) e web (Selenium) com excelente integração.
- Permite usar a mesma stack nos dois projetos — repositório e pipeline mais simples.
- Sintaxe enxuta: o mesmo teste em Java tem o dobro de linhas.

3.2 Por que pytest e não unittest?

- Sintaxe mais simples — usa o assert nativo do Python.
- Sistema de fixtures muito mais poderoso (escopos, dependências, autouse).
- Plugins (pytest-html, pytest-xdist, allure-pytest, pytest-rerunfailures).
- Markers para selecionar subconjuntos: `pytest -m smoke`.
- Parametrize nativo: roda o mesmo teste com várias entradas.

3.3 Por que requests para API?

- Biblioteca padrão de fato em Python para HTTP.
- Sintaxe limpa: `requests.get(url, json=payload)`.
- Suporta sessões (reaproveitamento de conexão e cookies).
- Integra naturalmente com jsonschema para validação de contrato.

3.4 Por que Selenium para web?

- Padrão da indústria há mais de 15 anos.
- WebDriver virou padrão W3C — funciona em todos os browsers.
- Tem alternativas modernas (Playwright, Cypress), mas Selenium ainda é o mais cobrado em entrevistas e o que a maioria das empresas usa.

Trade-off honesto: Playwright é mais rápido e tem auto-wait nativo. Se fosse um projeto novo em produção, eu consideraria. Para esse trabalho, Selenium foi escolhido por ser o padrão de mercado e por estar explícito no enunciado.

3.5 Por que GitHub Actions?

- Já vem integrado ao repositório — zero configuração externa.
- YAML simples, fácil de explicar na apresentação.
- Free tier generoso: 2.000 minutos/mês para repos privados, ilimitado para públicos.
- Marketplace de actions (setup-python, setup-chrome, etc.).
- Badges no README dão impacto visual imediato.

3.6 Por que Allure Report?

- Relatório visualmente profissional, com steps, anexos, gráficos.
- Anexa screenshots automaticamente — perfeito para depurar testes web.
- Categoriza por epic/feature/story (estrutura de business).
- Mantém histórico de execução (vê tendência de falhas ao longo do tempo).

4. Projeto API - Petstore

4.1 O que é a Petstore

A Swagger Petstore é uma API pública criada pela equipe do Swagger/OpenAPI como exemplo. Ela simula uma loja de pets e expõe três grupos de endpoints:

- **Pet:** CRUD de pets (criar, atualizar, buscar por ID, buscar por status, deletar).
- **Store:** inventário e pedidos (orders).
- **User:** usuários da loja (CRUD + login/logout).

Por ser pública e instável, é importante ter cleanup robusto e IDs grandes pra evitar colisão.

4.2 Estrutura do projeto

```
api/
├── clients/
│   ├── base_client.py    # classe-base com session, headers, log
│   ├── pet_client.py     # operações sobre /pet
│   ├── store_client.py   # operações sobre /store
│   └── user_client.py    # operações sobre /user
├── schemas/              # contratos JSON Schema
├── data/factories.py     # geração de payload com Faker
├── tests/
│   ├── conftest.py       # fixtures compartilhadas
│   ├── test_pet.py
│   ├── test_store.py
│   └── test_user.py
├── config.py             # carregamento do .env
├── pytest.ini            # markers e options do pytest
└── requirements.txt
```

4.3 Camadas e responsabilidades

Service/Client pattern. Os testes nunca falam diretamente com requests. Eles falam com PetClient, StoreClient, UserClient. Isso traz três benefícios:

- Se a URL ou o cabeçalho mudar, eu ajusto em UM lugar (o client).
- Os testes ficam legíveis: `pet_client.create(payload)` é mais claro que `requests.post(url, json=...)`.
- Reuso: os mesmos clients servem para qualquer teste novo.

4.3.1 BaseClient (clients/base_client.py)

Classe-base que centraliza:

- Sessão HTTP (`Session()` reaproveita conexão, é mais rápido).
- Headers padrão (`Content-Type: application/json`).

- Timeout em todas as requests.
- Log de cada request com método, URL, status e tempo.
- Métodos genéricos: get, post, put, delete.

4.3.2 Clients específicos

Cada um herda de BaseClient e adiciona métodos do seu domínio. Exemplo do PetClient:

```
class PetClient(BaseClient):
    BASE = "/pet"

    def create(self, payload: dict):
        return self.post(self.BASE, json=payload)

    def get_by_id(self, pet_id: int):
        return self.get(f"{self.BASE}/{pet_id}")

    def find_by_status(self, status: str):
        return self.get(f"{self.BASE}/findByStatus", params={"status": status})
```

Decisão importante: os métodos retornam o response inteiro (não só o JSON). Por quê? Porque o teste pode querer validar status_code, headers, tempo, body. Se retornássemos só o JSON, perderíamos informação.

4.4 Validação de contrato com JSON Schema

Toda resposta de sucesso é validada contra um schema. Exemplo:

```
PET_SCHEMA = {
    "type": "object",
    "required": ["id", "name", "photoUrls"],
    "properties": {
        "id": {"type": "integer"},
        "name": {"type": "string"},
        "status": {"type": "string", "enum": ["available", "pending", "sold"]},
        ...
    },
}
```

```
# No teste:
validate(instance=response.json(), schema=PET_SCHEMA)
```

Por que isso é poderoso: valida que a API mantém o contrato. Se o backend mudar o tipo de "id" de integer para string, o teste quebra. Isso evita o tipo de bug que só explode em produção.

4.5 Geração de massa com Faker

Em vez de hardcodar dados, usamos factories:

```
def build_pet(status: str = "available") -> dict:
    return {
        "id": random.randint(10**8, 10**9),
```

```

"name": fake.first_name(),
"category": {"id": random.randint(1, 100), "name": fake.word()},
"photoUrls": [fake.image_url()],
"tags": [{"id": 1, "name": fake.word()}],
"status": status,
}

```

Vantagens: cada execução tem dados diferentes (não há colisão), e os testes ficam isolados entre si.

Por que IDs gigantes (10^8 a 10^9)? Petstore é pública. Se eu usasse ID 1, alguém estaria rodando o mesmo teste e nossas execuções iriam colidir. IDs grandes praticamente eliminam essa chance.

4.6 Fixtures (conftest.py)

Fixtures fazem setup/teardown reutilizável. Temos quatro:

```

@pytest.fixture(scope="session")
def pet_client():
    return PetClient()          # 1 cliente para a sessão inteira

@pytest.fixture
def new_pet(pet_client):
    payload = build_pet()
    response = pet_client.create(payload)
    yield response.json()       # entrega o pet ao teste
    pet_client.delete_by_id(payload["id"]) # limpa depois

```

Conceito-chave: o yield divide o setup do teardown. Tudo antes do yield roda antes do teste; tudo depois roda depois — mesmo se o teste falhar.

4.7 Cenários cobertos

Recurso	Tipo	Cenários
Pet	CRUD + extras	Create, GetById, Update (PUT), Update (form), Delete, FindByStatus parametrizado, response time
Pet	Negativos	GET pet 0 (404), DELETE pet 0
Store	Felizes	Inventory, PlaceOrder, GetOrder, DeleteOrder
Store	Negativos	Order inexistente, ID inválido (negativo)
User	Felizes	Create, Get, Update, Delete, Login, Logout, CreateWithList
User	Negativos	GET user inexistente

4.8 Verbos HTTP - revisão rápida

Método	Uso	Idempotente?
--------	-----	--------------

GET	Buscar recurso	Sim — buscar duas vezes não muda o servidor
POST	Criar recurso	Não — criar duas vezes gera dois recursos
PUT	Atualizar inteiro	Sim — atualizar duas vezes resulta no mesmo estado
PATCH	Atualizar parcial	Geralmente sim
DELETE	Remover	Sim — deletar de novo dá 404, mas o estado final é o mesmo

4.9 Status codes mais cobrados

- **200 OK:** sucesso. É o status default em GETs.
- **201 Created:** recurso criado (POST). A Petstore usa 200 mesmo no create, o que é uma quirk.
- **204 No Content:** sucesso sem body de resposta (típico de DELETE).
- **400 Bad Request:** payload inválido.
- **401 Unauthorized:** falta autenticação.
- **403 Forbidden:** autenticado, mas sem permissão.
- **404 Not Found:** recurso não existe.
- **500 Internal Server Error:** erro do servidor (não é nossa culpa, mas precisamos detectar).

5. Projeto Web - SauceDemo

5.1 O que é o SauceDemo

Site de demonstração da Sauce Labs, criado especificamente para treino de automação. Tem usuários com comportamentos diferentes:

- **standard_user**: comportamento normal — fluxo feliz.
- **locked_out_user**: login bloqueado — para testes negativos.
- **problem_user**: imagens trocadas — para testar visual.
- **performance_glitch_user**: respostas lentas — para testes de performance.

Senha sempre é "secret_sauce".

5.2 Estrutura do projeto

```
web/
├── pages/
│   ├── base_page.py      # esperas, click, type, find genéricos
│   ├── login_page.py
│   ├── inventory_page.py
│   ├── cart_page.py
│   └── checkout_page.py
├── tests/
│   ├── conftest.py       # fixtures + hook de screenshot
│   ├── test_login.py
│   ├── test_cart.py
│   └── test_checkout.py
├── utils/driver_factory.py # cria driver Chrome ou Firefox
├── config.py
├── pytest.ini
└── requirements.txt
```

5.3 Page Object Model (POM)

Padrão de design que diz: cada tela do sistema vira uma classe. Essa classe expõe ações de alto nível (login, addToCart) e esconde detalhes de baixo nível (seletores CSS, esperas).

Sem POM (ruim):

```
def test_login(driver):
    driver.find_element(By.ID, "user-name").send_keys("standard_user")
    driver.find_element(By.ID, "password").send_keys("secret_sauce")
    driver.find_element(By.ID, "login-button").click()
    assert driver.current_url.endswith("/inventory.html")
```

Com POM (bom):

```
def test_login(login_page, driver):
    login_page.login("standard_user", "secret_sauce")
    assert InventoryPage(driver).is_loaded()
```

Vantagens:

- Se o ID do botão de login mudar, eu corrijo em UM lugar (LoginPage), não em 50 testes.
- Os testes lêem como uma história de usuário — perfeito pra mostrar pra um cliente.
- Reuso natural: a mesma LoginPage serve para qualquer teste que precise logar.

5.4 BasePage - a classe-mãe

Centraliza esperas e ações comuns para que nenhuma página precise reimplementar:

```
class BasePage:
    def __init__(self, driver):
        self.driver = driver
        self.wait = WebDriverWait(driver, Config.EXPLICIT_WAIT)

    def find(self, locator):
        return self.wait.until(EC.presence_of_element_located(locator))

    def click(self, locator):
        self.wait.until(EC.element_to_be_clickable(locator)).click()

    def type(self, locator, text: str):
        element = self.find(locator)
        element.clear()
        element.send_keys(text)
```

Decisão crítica: ZERO time.sleep. Tudo é WebDriverWait com expected conditions. Sleeps são a causa #1 de testes flaky.

5.5 Tipos de espera (waits)

Tipo	Comportamento
Implicit wait	Configurada uma vez no driver. Toda busca de elemento espera até X segundos. Difícil de debugar.
Explicit wait	WebDriverWait + ExpectedConditions. Espera por uma condição específica (visible, clickable, presence). É o que usamos.
Fluent wait	Variação avançada do explicit, com poll customizado e ignore de exceptions. Não precisamos.
time.sleep	NUNCA usar. Sempre falha em CI lento ou passa em CI rápido — origem da flakiness.

5.6 Locators (estratégias de busca)

Selenium oferece várias formas de localizar elementos. Ordem de preferência:

- **ID:** mais rápido e estável (quando existe).

- **Name:** estável, comum em formulários.
- **CSS Selector:** flexível, rápido. Padrão moderno.
- **Class Name:** cuidado se o classe é compartilhada.
- **Link Text / Partial Link Text:** só serve para <a>, e quebra se o texto mudar.
- **XPath:** mais poderoso, mas mais frágil. Usar como último recurso.

Boa prática: priorizar atributos data-test (como o SauceDemo faz com data-test="error"), pois são imunes a refactor visual.

5.7 DriverFactory

Centraliza a criação do driver. Suporta Chrome ou Firefox, headless ou visual:

```
class DriverFactory:
    @staticmethod
    def create():
        browser = Config.BROWSER.lower()
        if browser == "chrome":
            options = ChromeOptions()
            if Config.HEADLESS:
                options.add_argument("--headless=new")
                options.add_argument("--window-size=1920,1080")
            return webdriver.Chrome(
                service=ChromeService(ChromeDriverManager().install()),
                options=options
            )
```

webdriver-manager baixa automaticamente o ChromeDriver compatível com a versão instalada — zero configuração manual.

5.8 Hook de screenshot em falha

No conftest.py temos um hook do pytest que tira screenshot automático quando um teste falha:

```
@pytest.hookimpl(hookwrapper=True)
def pytest_runtest_makereport(item, call):
    outcome = yield
    report = outcome.get_result()
    if report.when == "call" and report.failed:
        drv = item.funcargs.get("driver")
        if drv:
            filename = SCREENSHOT_DIR / f"{item.name}_{timestamp}.png"
            drv.save_screenshot(str(filename))
            allure.attach(open(filename, "rb").read(), ...)
```

Por que isso é importante: quando um teste falha em CI, você não está olhando o navegador. O screenshot é a única pista visual do que estava acontecendo. Anexar no Allure deixa o debug muito mais rápido.

5.9 Cenários web cobertos

Feature	Tipo	Cenários
Login	Felizes	standard_user, logout
Login	Negativos	locked_out_user, senha errada, campos vazios
Carrinho	Felizes	Adicionar 1 item, adicionar 3 itens, remover, persistir entre páginas, ordenar por preço crescente
Checkout	Felizes	Fluxo E2E completo (smoke)
Checkout	Negativos	Sem nome, sem CEP

6. CI/CD com GitHub Actions

6.1 O que é CI/CD

- **CI (Continuous Integration):** a cada commit, o sistema constrói e testa o código automaticamente. Detecta problemas cedo.
- **CD (Continuous Delivery/Deployment):** o código aprovado é colocado automaticamente em ambiente de homologação ou produção.

Nosso trabalho cobre CI (testes automáticos a cada commit). Não temos CD porque não há aplicação para deployar.

6.2 Anatomia de um workflow do GitHub Actions

```
name: API Tests - Petstore      # nome legível
on:                             # quando rodar
  push:
    branches: [main, develop]
    paths: ['api/**']          # só dispara se mudou algo em api/
  pull_request:
    branches: [main]
  schedule:
    - cron: '0 7 * * 1'        # toda segunda às 7h UTC
  workflow_dispatch:           # botão manual

jobs:
  api-tests:                   # nome do job
    runs-on: ubuntu-latest     # SO da máquina
    strategy:
      matrix:
        python-version: ['3.11', '3.12'] # roda em 2 versões
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
      - with:
          python-version: ${ { matrix.python-version } }
          cache: 'pip'
      - run: pip install -r api/requirements.txt
      - run: cd api && pytest --alluredir=allure-results
      - uses: actions/upload-artifact@v4
      - if: always()
      - with:
          name: api-allure-results
          path: api/allure-results
```

6.3 Conceitos do workflow

- **Trigger:** o evento que dispara o workflow (push, pull_request, schedule, workflow_dispatch).
- **Job:** uma unidade de trabalho que roda em uma máquina virtual.

- **Step:** um comando dentro do job (instalar, rodar testes, fazer upload).
- **Runner:** a máquina virtual onde o job roda. Aqui é ubuntu-latest.
- **Action:** um step reutilizável feito por terceiros. Ex.: actions/checkout@v4 baixa o código.
- **Matrix strategy:** roda o mesmo job com várias combinações (Python 3.11 e 3.12).
- **Artifact:** arquivo gerado pelo job que fica disponível pra download depois.

6.4 Os 3 workflows do projeto

- **api-tests.yml:** roda os testes da API em matriz Python 3.11 + 3.12. Triggers: push, PR, agendado segunda 7h, manual.
- **web-tests.yml:** roda os testes Web em Chrome headless. Sobe o Chrome via setup-chrome action. Faz upload dos screenshots em caso de falha.
- **lint.yml:** valida estilo de código com flake8.

6.5 Por que separar em workflows distintos?

- Independência: se eu mudar só código de API, o workflow web não roda — economiza minutos.
- Paths filter: usei paths: [api/**] e paths: [web/**] para isso.
- Logs separados: fica óbvio o que quebrou.
- Cada um pode ter um schedule diferente no futuro.

7. pytest a fundo

7.1 Discovery (descoberta de testes)

pytest descobre testes automaticamente seguindo o padrão configurado em `pytest.ini`:

```
testpaths = tests
python_files = test_*.py
python_classes = Test*
python_functions = test_*
```

Logo: arquivos `test_*.py`, dentro deles classes `Test*`, dentro delas métodos `test_*`. Tudo descoberto sozinho.

7.2 Fixtures e seus escopos

Fixture é uma função decorada com `@pytest.fixture`. Os escopos definem quantas vezes ela roda:

- **function (default):** uma vez por teste.
- **class:** uma vez por classe.
- **module:** uma vez por arquivo.
- **session:** uma vez por execução inteira do pytest.

Decisão do projeto: os clients (`PetClient`, etc.) têm escopo `session` — não precisa criar um por teste. Já as fixtures que criam dados (`new_pet`, `new_user`) têm escopo `function` — cada teste tem seus próprios dados isolados.

7.3 Markers

Tags que classificam testes. Permitem rodar subconjuntos:

```
@pytest.mark.smoke
@pytest.mark.pet
def test_create_pet(pet_client):
    ...
```

```
# Linha de comando:
pytest -m smoke          # só smoke
pytest -m "smoke and pet" # combinações
pytest -m "not negative"  # exclusão
```

7.4 Parametrize

Roda o mesmo teste com várias entradas:

```
@pytest.mark.parametrize("status", ["available", "pending", "sold"])
def test_find_by_status(pet_client, status):
    response = pet_client.find_by_status(status)
    assert response.status_code == 200
```

No relatório aparece como 3 testes diferentes: `test_find_by_status[available]`, `[pending]` e `[sold]`.

7.5 `conftest.py`

Arquivo especial onde fixtures e hooks ficam disponíveis automaticamente para todos os testes na pasta. Não precisa importar.

7.6 Hooks

Pontos de extensão do pytest. Usamos `pytest_runtest_makereport` para tirar screenshot em falha.

8. Selenium a fundo

8.1 Como o Selenium funciona

Arquitetura em três peças:

- **Cliente Selenium (nosso código Python):** envia comandos via protocolo W3C WebDriver.
- **Driver (ChromeDriver, GeckoDriver):** processo intermediário que converte os comandos em ações do browser.
- **Browser (Chrome, Firefox):** quem efetivamente roda os comandos.

Fluxo: Python → JSON via HTTP → ChromeDriver → DevTools Protocol → Chrome.

8.2 Modo headless

Roda o browser sem abrir a janela. Vantagens:

- Mais rápido (sem renderizar pixels).
- Roda em servidor sem display (CI).
- Menos consumo de recursos.

Quando NÃO usar: quando você está debugando localmente. É bom ver o que está acontecendo.

8.3 Por que webdriver-manager?

Antigamente você baixava o ChromeDriver na mão e colocava no PATH. Toda vez que o Chrome atualizava, o driver quebrava. webdriver-manager resolve: detecta a versão do Chrome instalado e baixa o driver compatível na hora.

8.4 Causas comuns de teste flaky

- Sleeps fixos (time.sleep) — se a rede está lenta, falha.
- Locators frágeis (XPath super específico).
- Animations/transitions sem espera adequada.
- Estado compartilhado entre testes (não limpou cookies).
- Race conditions (clicar antes do elemento estar realmente clicável).

9. Padrões de design adotados

9.1 Page Object Model (POM)

Já explicado no capítulo 5. Resumo de uma frase: cada tela vira uma classe que esconde os detalhes do HTML.

9.2 Service / Client pattern

Análogo do POM para API. Cada recurso (Pet, Store, User) tem uma classe que encapsula chamadas HTTP. Os testes nunca falam direto com requests.

9.3 Factory pattern

Em vez de hardcodar payloads, temos funções que constroem dados frescos com Faker. Isso garante que os testes são independentes entre si e não colidem em ambientes compartilhados.

9.4 Singleton (informal) - DriverFactory

Centraliza a criação do driver. Se um dia eu quiser trocar Chrome por Firefox, mudo em um lugar só.

9.5 Princípios SOLID e DRY presentes

- **Single Responsibility:** cada classe tem um propósito (PetClient só fala de Pet, LoginPage só lida com login).
- **Open/Closed:** para adicionar um novo recurso, eu crio um novo client, sem mexer nos existentes.
- **DRY (Don't Repeat Yourself):** BasePage e BaseClient eliminam duplicação.

10. 30 perguntas que a professora pode fazer

1. Por que escolheu Python e não Java?

Resposta: Pela sintaxe mais limpa (testes em pytest têm metade das linhas de TestNG/JUnit), pelo ecossistema (pytest é referência), e porque permitiu usar a mesma stack na API e no Web — repositório e pipeline mais simples.

2. Por que usar Page Object Model?

Resposta: Para isolar mudanças. Se um botão muda de ID, eu altero em uma classe (LoginPage) e todos os testes que usam ela continuam funcionando. Sem POM, eu teria que caçar o seletor em todos os testes.

3. O que é um teste flaky e como você evita?

Resposta: Teste que às vezes passa e às vezes falha sem mudança de código. Evito não usando `time.sleep` — uso `WebDriverWait` com `expected conditions`. Também limpo dados criados em fixtures (`yield + delete`) para não vazar estado entre testes.

4. Diferença entre POST e PUT?

Resposta: POST cria um recurso (não-idempotente: chamar duas vezes cria dois). PUT atualiza um recurso por inteiro (idempotente: chamar duas vezes deixa no mesmo estado).

5. O que é idempotência?

Resposta: Uma operação é idempotente quando executá-la N vezes tem o mesmo efeito que executá-la uma vez. GET, PUT, DELETE são idempotentes. POST não é.

6. Por que validar JSON Schema?

Resposta: Para garantir contrato. Se o backend mudar o tipo de "id" de integer pra string sem avisar, o teste pega antes de explodir em produção.

7. O que faria diferente em produção?

Resposta: Adicionaria autenticação real (OAuth/JWT), retry com backoff em testes flaky, paralelização com `pytest-xdist` em workers separados, e um stage de deploy de relatórios no GitHub Pages.

8. Por que separou em dois projetos no mesmo repo?

Resposta: Para cada projeto ter `requirements.txt`, `pytest.ini` e pipeline próprios — mas ainda compartilhando docs, README e Makefile. Best of both worlds.

9. Como você pensou na cobertura de testes?

Resposta: Para a API: CRUD completo de cada recurso + pelo menos um cenário negativo + validação de schema. Para o Web: fluxo E2E principal (login → carrinho → checkout) + variações negativas em cada etapa. Estratégia documentada em TEST_STRATEGY.md.

10. Por que Allure Report?

Resposta: Por ser visualmente profissional, anexar screenshots automaticamente, organizar por epic/feature/story e manter histórico. Em entrevista de QA, Allure é cobrado.

11. O que é o conftest.py?

Resposta: Arquivo especial do pytest onde fixtures e hooks ficam disponíveis automaticamente para todos os testes na pasta. Não precisa importar.

12. Como funciona o screenshot em falha?

Resposta: Tem um hook `pytest_runtest_makereport` no `conftest.py` do projeto web. Quando um teste falha (`report.failed`), pego o driver da fixture, salvo PNG e anexo no Allure.

13. O que é fixture com escopo session?

Resposta: Roda uma vez por execução inteira. Usado para os clients de API — não preciso criar um `PetClient` novo a cada teste.

14. Diferença entre implicit e explicit wait?

Resposta: Implicit: configurada uma vez no driver, toda busca espera até X seg. Explicit: `WebDriverWait` + `ExpectedConditions`, espera por condição específica. Eu uso explicit porque é mais previsível e deixa o teste mais legível.

15. Por que não usar XPath?

Resposta: XPath funciona, mas é mais frágil. Prefiro ID, name, CSS selector. XPath fica como último recurso.

16. O que é matrix no GitHub Actions?

Resposta: Uma estratégia que roda o mesmo job com várias combinações de variáveis. No projeto, rodo a API em Python 3.11 e 3.12 simultaneamente — pega problemas de compatibilidade.

17. O que é um artifact?

Resposta: Arquivo gerado pelo job que fica disponível pra download por um tempo (14 dias no nosso caso). Eu uso para guardar o report HTML, os resultados Allure e os screenshots de falha.

18. Como você lidaria com autenticação OAuth?

Resposta: Criaria uma fixture de session scope que faz o flow OAuth uma vez, guarda o token, e injeta no header de todas as requests via `session.headers.update`.

19. Por que a Petstore é difícil de testar?

Resposta: É pública e instável — outras pessoas estão criando dados ao mesmo tempo. Por isso eu uso IDs grandes (10^8 a 10^9) pra evitar colisão e fixtures com cleanup pra não deixar lixo.

20. O que é flake8?

Resposta: Linter de Python que verifica estilo (PEP 8). Roda no workflow `lint.yml`. Garante consistência de código.

21. Como rodar um teste específico?

Resposta: `pytest tests/test_pet.py::TestPet::test_create_pet`, ou por marker: `pytest -m smoke`.

22. O que é parametrize?

Resposta: Decorator que roda o mesmo teste com várias entradas. No relatório aparece como N testes distintos.

23. Como fazer testes paralelos?

Resposta: Com `pytest-xdist`: `pytest -n 4` roda em 4 workers. Web exige cuidado: cada worker tem seu próprio driver.

24. O que faz o `requests.Session()`?

Resposta: Reaproveita a conexão TCP entre requests, persiste cookies e headers. É mais rápido e mais realista.

25. Diferença entre 401 e 403?

Resposta: 401 = não autenticado (não passou credencial). 403 = autenticado, mas sem permissão.

26. O que é `assertion message`?

Resposta: Texto que aparece quando o `assert` falha. Ex.: `assert x == 1, "x deveria ser 1, recebeu " + str(x)`. Facilita debug.

27. Por que três workflows e não um só?

Resposta: Independência. Se mudar só código da API, o workflow web não roda — economiza minutos. Filtro por paths.

28. Como você escalaria isso para 1000 testes?

Resposta: Paralelização com xdist, dividir em smoke/regression (smoke roda em PR, regression à noite), cache de dependências, runners auto-hospedados se preciso de mais poder.

29. O que é TDD e BDD?

Resposta: TDD = Test-Driven Development: escreve teste antes do código. BDD = Behavior-Driven Development: escreve teste em linguagem de negócio (Given/When/Then), tipicamente com Cucumber/Gherkin.

30. O que vai melhorar nesse projeto se tiver mais tempo?

Resposta: Adicionaria testes BDD (pytest-bdd), Docker para rodar o ambiente em container, métricas de tempo de execução com pytest-benchmark, e cross-browser real com Selenium Grid.

11. Roteiro da apresentação (15 minutos)

11.1 Estrutura sugerida

Tempo	Bloco	O que fazer
0:00 - 1:30	Abertura	Apresentar-se, o que foi pedido, o que foi entregue (1 frase de cada). Mostrar o GitHub do projeto.
1:30 - 4:00	Visão geral	Mostrar README com o diagrama Mermaid. Explicar a estrutura: api/, web/, .github/workflows/. Falar da decisão de repo único.
4:00 - 7:30	API ao vivo	Abrir api/clients/base_client.py e api/tests/test_pet.py. Mostrar 1 teste happy + 1 teste negativo. Rodar pytest -m smoke localmente OU mostrar a pipeline rodando.
7:30 - 11:00	Web ao vivo	Abrir web/pages/login_page.py e web/tests/test_checkout.py. Explicar Page Object Model. Rodar o test_full_checkout em modo visual (com o navegador abrindo).
11:00 - 13:00	CI/CD	Mostrar GitHub Actions: o histórico de execuções, badges no README, artifacts, screenshots de falha (se tiver).
13:00 - 14:00	Allure Report	Abrir o relatório Allure (pré-gerado). Mostrar a categorização por feature, os steps de cada teste, screenshot.
14:00 - 15:00	Encerramento	Recap rápido (cobertura, padrões usados). Falar 1-2 melhorias futuras. Abrir para perguntas.

11.2 Frases-chave para usar

- "Optei por separar em camadas para que mudanças no contrato da API ou no HTML não exijam reescrever os testes."
- "Cada teste é independente — cria seus dados na fixture e limpa no teardown."
- "Não uso time.sleep em nenhum teste — apenas esperas explícitas."
- "A pipeline roda em duas versões de Python para detectar problemas de compatibilidade."
- "O screenshot é anexado automaticamente no Allure quando um teste web falha — facilita o debug em CI."

11.3 O que ter aberto antes de começar

- Aba 1: GitHub do repositório (README na raiz).
- Aba 2: GitHub Actions (histórico de execuções).
- Aba 3: Relatório Allure pré-gerado em local.
- Aba 4: VS Code com o projeto aberto.

- Aba 5: Terminal pronto pra rodar pytest.

11.4 O que NÃO fazer

- Não falar muito tempo de teoria — é prático, mostra código.
- Não rodar a suite inteira (vai demorar) — mostre 2 testes.
- Não improvisar quando perguntada — se não souber, diga "boa pergunta, eu pesquisaria mais sobre isso".
- Não esquecer de testar o setup ANTES — se a internet falhar na hora, tenha um vídeo gravado de backup.

12. Glossário de termos

Termo	Definição
Action (GitHub Actions)	Step reutilizável publicado por terceiros (ex.: actions/checkout).
Allure Report	Framework de relatórios de teste com UI interativa, screenshots e histórico.
Artifact	Arquivo gerado pelo workflow disponível para download depois.
Assertion (assert)	Verificação que decide se um teste passou ou falhou.
BasePage / BaseClient	Classes-mãe com métodos comuns (clique, esperar, etc.) reutilizados em todas as páginas/clients.
Caixa preta (Black box)	Teste que não conhece o código interno, só entrada e saída.
CI (Continuous Integration)	Build e teste automáticos a cada commit.
Cleanup / Teardown	Etapas de limpeza após o teste (deletar dados criados, fechar driver).
Conftest.py	Arquivo do pytest onde fixtures e hooks ficam disponíveis automaticamente.
Cron	Sintaxe para agendar execução periódica (ex.: 0 7 * * 1 = toda segunda 7h UTC).
Driver (WebDriver)	Componente que traduz comandos Selenium em ações do browser.
E2E (End-to-end)	Teste que percorre o fluxo completo do usuário, da UI até o banco.
Faker	Biblioteca para gerar dados falsos realistas (nomes, emails, etc.).
Fixture	Função que prepara estado antes do teste e limpa depois.
Flaky	Teste instável — passa às vezes, falha às vezes, sem mudança de código.
Headless	Browser rodando sem interface gráfica.
Hook (pytest)	Ponto de extensão do pytest que permite injetar lógica em fases do ciclo.
Idempotência	Operação que produz o mesmo efeito se chamada N vezes ou 1 vez.
JSON Schema	Especificação formal do formato de um JSON, usada para validar contrato.
Job (workflow)	Unidade de trabalho em um workflow do GitHub Actions.
Linter	Ferramenta que valida estilo de código (ex.: flake8, black, eslint).
Locator	Estratégia para localizar elemento na página (ID, CSS, XPath).

Marker	Tag em testes do pytest que permite seleção (-m smoke).
Matrix strategy	Rodar o mesmo job com várias combinações (ex.: Python 3.11 e 3.12).
Mock	Objeto falso que substitui dependência real para isolar o teste.
Negative test	Teste de cenário de erro (404, campo vazio, valor inválido).
Page Object Model (POM)	Padrão onde cada tela vira uma classe que esconde detalhes de implementação.
Parametrize	Decorator do pytest que roda o mesmo teste com várias entradas.
Path filter	Configuração que dispara workflow apenas se arquivos específicos mudaram.
Pirâmide de testes	Modelo de proporção: muitos unit, médios de integração, poucos E2E.
POM	Sigla para Page Object Model.
Pytest	Framework de testes em Python, mais usado que unittest.
Regression	Suite completa que verifica que nada quebrou.
REST	Estilo arquitetural para APIs HTTP usando verbos (GET, POST, etc.).
Runner	Máquina virtual que executa o workflow (ubuntu-latest, windows-latest).
Schema validation	Validar formato da resposta contra um JSON Schema.
Selenium	Biblioteca para automação de browser via WebDriver.
Service / Client pattern	Análogo do POM para API: cada recurso vira uma classe.
Smoke test	Teste mínimo que valida que o sistema "sobe".
Step	Comando individual dentro de um job.
Suite	Conjunto de testes.
Trigger	Evento que dispara um workflow (push, pull_request, schedule).
Wait (explicit / implicit)	Mecanismo do Selenium para aguardar elemento ficar disponível.
Workflow	Arquivo YAML com definição de pipeline no GitHub Actions.
XPath	Linguagem de query para XML/HTML, usada como locator (último recurso).

13. Dashboard e grafo de topologia

13.1 O que é

Além das suítes de teste, o repositório tem uma página HTML de dashboard (docs/dashboard.html) que serve como capa visual do projeto. Nela existe um elemento central: um grafo 3D estilo Obsidian que representa a topologia real do projeto.

O grafo não é decorativo. Cada nó (ponto colorido) é um arquivo ou módulo real do projeto. Cada aresta (linha) é uma dependência real entre eles - import, herança, ou relação de execução.

13.2 Composição do grafo

29 nós distribuídos em 4 clusters de cores:

Cluster	Cor	Conteúdo
Root	Branco	repo (raiz do repositório)
API	Âmbar	13 nós: api/, BaseClient, PetClient, StoreClient, UserClient, schemas, factories, conftest, e os 3 test files
Web	Verde sage	11 nós: web/, BasePage, 4 page objects, DriverFactory, conftest, e os 3 test files
CI	Cyan	4 nós: .github/workflows/ + os 3 yml de pipeline

Total: 47 arestas conectando os nós. Por exemplo: PetClient herda de BaseClient (aresta), test_pet.py importa o pet_schema (aresta), api-tests.yml executa os 3 test files de api (3 arestas).

13.3 Como funciona (force-directed)

O grafo não é desenhado com posições fixas. Ele se organiza sozinho usando uma simulação física rodando 60 vezes por segundo. Quatro forças atuam simultaneamente em cada nó:

- **Repulsão:** todos os nós se empurram entre si (força inversamente proporcional ao quadrado da distância). Isso impede que fiquem amontoados.
- **Atração (mola):** nós conectados por aresta se puxam (força tipo mola, com comprimento de repouso). Isso faz com que nós relacionados fiquem perto.
- **Centralização:** uma força fraca puxa todo nó na direção do centro. Impede que o grafo "exploda" pra fora da tela.
- **Amortecimento (damping):** a velocidade de cada nó é multiplicada por 0.86 a cada quadro. Faz o sistema convergir, em vez de oscilar pra sempre.

Resultado: o grafo encontra um equilíbrio onde nós muito conectados ficam centrais e nós periféricos se afastam. É exatamente o que o Obsidian faz com seu graph view.

13.4 Tecnologia

- **Three.js (r128):** biblioteca de renderização WebGL. Carregada via CDN do Cloudflare.
- **WebGL:** API do navegador pra renderizar 3D acelerado por GPU.
- **Raycaster:** utilitário do Three.js que detecta qual objeto 3D o mouse está apontando — usado pra mostrar o tooltip no hover.
- **Sprites:** cada nó tem uma esfera + uma sprite (textura 2D que sempre encara a câmera) pra dar efeito de halo/glow.

13.5 Interações disponíveis

- **Drag (clicar e arrastar):** rotaciona a câmera ao redor do grafo, em qualquer direção.
- **Hover (passar o mouse):** o nó aumenta, o halo intensifica, e um tooltip mostra a categoria (API/WEB/CI/ROOT) e o nome do arquivo.
- **Scroll:** zoom in/out (entre 5 e 20 unidades de distância).
- **Auto-rotação:** quando o mouse não está interagindo, a câmera rotaciona suavemente sozinha. Para após o primeiro click.

13.6 Como defender na apresentação

Quando a professora perguntar o que é o grafo, sua resposta deve ser:

"Esse é o grafo de topologia do projeto, no estilo do Obsidian. Cada nó é um arquivo real do código — clientes HTTP, page objects, schemas, factories, testes e workflows. Cada linha é uma dependência real entre eles, seja por import, herança ou execução."

Se ela perguntar como funciona:

"É um grafo force-directed. Quatro forças físicas atuam ao mesmo tempo: repulsão (afasta tudo), molas (puxam o que está conectado), centralização (mantém no centro) e amortecimento (estabiliza). O resultado é uma organização emergente — quem é mais conectado fica no meio, quem é menos conectado fica na periferia."

Se ela perguntar por que fez isso:

"Pra mostrar visualmente o que o código faz. Em vez de só listar os arquivos no README, eu queria que ficasse evidente como eles se relacionam. Quando você passa o mouse no nó test_checkout, vê

que ele depende de quatro page objects, um conftest e um workflow — isso conta a história do código melhor que qualquer texto."

13.7 Perguntas avançadas que podem aparecer

Como você decidiu o que é nó e o que não é?

Cada arquivo Python relevante virou um nó (clients, pages, schemas, conftests, factories, tests). Os módulos (api/, web/, .github/) viram nós-âncora dos clusters. Não inclui arquivos triviais como `__init__.py`, `requirements.txt`, `.gitignore` — seriam ruído visual.

Como você decidiu as conexões?

São conexões reais do código. Por exemplo: `PetClient` herda de `BaseClient` → aresta. `test_pet.py` usa o `factories.py` → aresta. `api-tests.yml` executa os test files → 3 arestas. Eu mapeei tudo manualmente lendo o código.

E se eu adicionar um novo teste, o grafo se atualiza sozinho?

Não automaticamente. Os nós e arestas estão hardcoded no JavaScript do dashboard. Pra um projeto maior em produção, eu poderia escrever um script Python que faz parsing dos imports e exporta um JSON, e o dashboard leria esse JSON. Mas pra escala desse trabalho, manter manual é mais simples.

Por que 3D e não 2D?

2D ficaria mais legível pra leitura técnica, mas 3D dá impacto visual e permite ver a topologia de ângulos diferentes ao rotacionar. Como é um trabalho acadêmico onde o objetivo é demonstrar capacidade técnica e atenção a detalhe, 3D somou mais que 2D somaria.

Tem algum trade-off?

Tem. WebGL exige um navegador moderno, e a página é mais pesada. Em telas pequenas o grafo fica menos legível. Em produção, eu provavelmente daria opção de alternar entre visualização 3D e 2D, e geraria a partir de uma análise estática do código em vez de hardcoded.

13.8 Caso a professora queira ver o código

O código do grafo está em `docs/dashboard.html`, dentro da segunda tag `<script>`. Os pontos-chave pra mostrar:

- A array `NODES` com os 29 nós, cada um com `id`, `label`, `categoria` e `tamanho`.
- A array `EDGES` com os 47 pares de conexões.
- A função `step()` que aplica as 4 forças por quadro.
- A função `animate()` que orquestra simulação + render + câmera orbital.

- O Raycaster pra detecção de hover.

Boa apresentação!

Lembra: você fez o trabalho, você sabe defender. Confie no que está aqui.

Se travar em alguma pergunta, respira fundo, olha pro código e responde com calma. Ninguém sabe tudo, e mostrar honestidade técnica é melhor que improvisar errado.

Boa sorte!